

Sprint Planner — Funcionamento Completo do Projeto

Documentação técnica do alto nível aos detalhes de implementação.

1. O QUE É

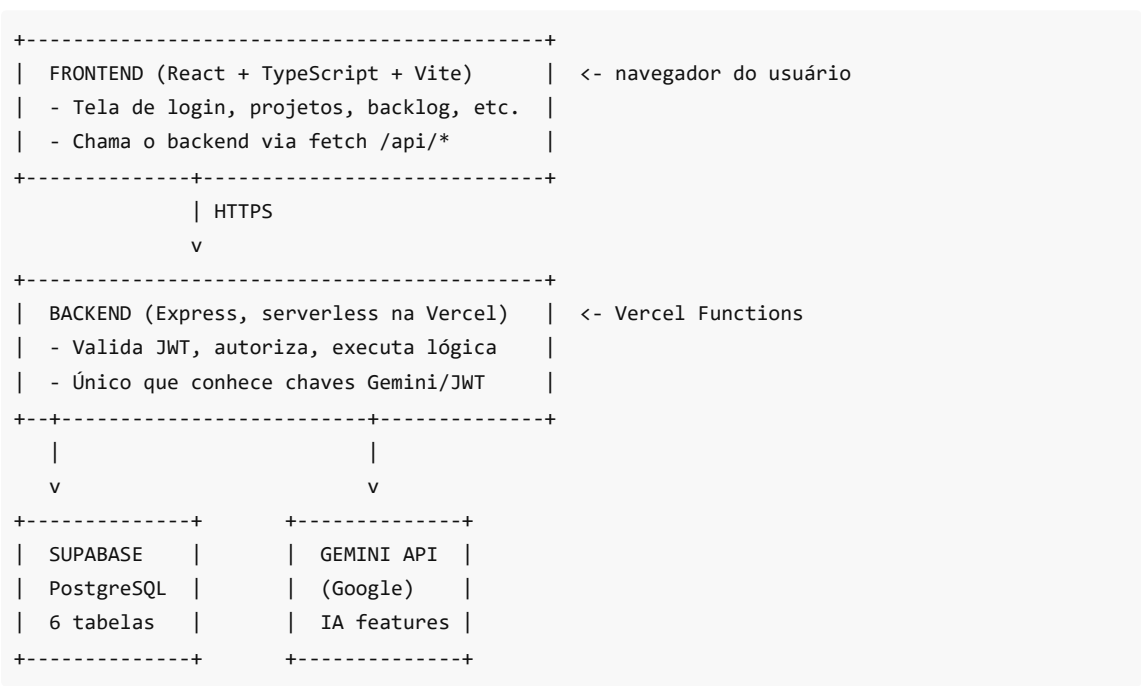
Sprint Planner é uma aplicação web para times pequenos gerenciarem projetos no método **Scrum**, com IA integrada (Google Gemini) para acelerar o planejamento.

Equivalente simplificado do Jira/Linear/Trello, mas:

- Mais enxuto (foco no essencial)
- Com IA gerando histórias e analisando sprints
- Construído usando IA como copiloto (Claude Code)

URL produção: <https://sprint-planner-murex.vercel.app>

2. ARQUITETURA EM 3 CAMADAS



Por que assim: o frontend nunca toca em senhas, chaves ou banco direto. Tudo passa pelo backend que valida quem você é (JWT) e o que você pode fazer (membership do projeto).

3. BANCO DE DADOS — 6 Tabelas

Todas no Supabase (PostgreSQL na nuvem) com prefixo `sp_` :

Tabela	O que guarda	Relações
<code>sp_users</code>	username, email, hash da senha	base de tudo

sp_projects	name, description, owner_id	tem dono (User)
sp_project_members	project_id, user_id, role (PO/SM/Dev)	N:N entre user e project
sp_invitations	inviter, invitee, status (pending/accepted/rejected)	convites pendentes
sp_sprints	name, goal, dates, status (planning/active/completed)	um projeto tem N sprints
sp_user_stories	title, description, points, label, status, sprint_id, assignee_id	cards do backlog

Regras no banco (defesa em profundidade):

- `FOREIGN KEY ... ON DELETE CASCADE` — deletar projeto remove sprints/stories/membros automaticamente
- `CHECK(role IN ('PO', 'Scrum Master', 'Dev'))` — banco rejeita roles inválidos
- `CHECK(status IN ('to_do', 'in_progress', 'in_review', 'done'))` — status do Kanban
- `UNIQUE(project_id, user_id)` — não dá pra adicionar mesmo membro 2x

4. SEGURANÇA — Como o usuário é autenticado

Registro (`POST /api/auth/register`)

1. User envia `{ username, email, password }`
2. Backend faz `bcrypt.hashSync(password, 10)` — hash irreversível
3. Insere no `sp_users` com **hash**, nunca a senha pura
4. Gera JWT assinado com `JWT_SECRET` contendo `{ id, username }`
5. Retorna `{ token, user }` pro frontend

Login (`POST /api/auth/login`)

1. User envia `{ email, password }`
2. Backend busca user no banco
3. `bcrypt.compareSync(senhaPura, hashDoBanco)` — true se bate
4. Gera novo JWT, retorna

Requisições protegidas

1. Frontend salva token no `localStorage`
2. Cada fetch envia `Authorization: Bearer <token>`
3. **AuthMiddleware** no backend:
 - Valida assinatura JWT
 - Verifica que user ainda existe no banco (cache 60s)
 - Anexa `req.user = { id, username }` para a rota usar

Autorização (anti-IDOR)

Toda rota protegida chama `isMember(projectId, userId)` antes de qualquer operação. Exemplo:

- "Mover história do projeto A para sprint do projeto B"? → 403
- "Ver membros de projeto que não participo"? → 403
- "Atribuir tarefa a alguém que não é membro"? → 400

5. FLUXOS PRINCIPAIS

Fluxo 1: Cadastro e primeiro projeto

```
Tela Login -> "Criar conta"
  v
Tela Register -> preenche, cria conta, recebe JWT
  v
Lista de Projetos (vazia) -> "+ Novo Projeto"
  v
Wizard de criação:
  1. Form: nome + descricao
  2. Chat com IA (3-4 perguntas)
  3. IA gera 4-6 historias
  4. User escolhe quais ir pro backlog
  5. Cria projeto + historias em batch
  v
Workspace do projeto (tabs: Backlog | Sprints | Board)
```

Fluxo 2: Convidar membro

```
Workspace -> icone Membros (canto superior direito)
  v
MembersModal: lista membros + form "Convidar"
  v
Owner digita "alice@email.com" + role "Dev" -> Convidar
  v
Backend: cria invitation status='pending'
  v
Alice (logada) ve sino com badge "1" no header
  v
Alice abre painel -> Aceitar
  v
Backend: cria sp_project_members + marca invitation accepted
  v
Alice agora ve o projeto na lista dela
```

Fluxo 3: Sprint completo (do backlog ao Kanban)

```
Tab Backlog -> cria historias (manualmente ou Gerar com IA)
  v
Tab Sprints -> "+ Novo Sprint" -> preenche datas + meta
  v
Sprint card -> "Iniciar" -> status muda para 'active'
  v
Volta no Backlog -> cada card tem <select> "Mover para sprint"
  Escolhe "Sprint Demo (ativo)"
  v
Tab Board -> 4 colunas (A fazer | Em andamento | Em revisao | Concluido)
  Drag & drop card entre colunas (UI atualiza ANTES da API = optimistic)
```

```

    v
Click no card -> painel slide-out a direita
  Edita titulo, descricao, criterios inline (auto-save no blur)
  Atribui responsavel (avatar colorido aparece)
    v
Sprint terminou -> Tab Sprints -> "Resumo IA"
  IA analisa todas as historias -> markdown com Entregas/Gargalos/Sugestoes
    v
Sprint card -> "Concluir" -> status 'completed'
```

6. IA NO PRODUTO — 3 Features Detalhadas

Feature A: Wizard conversacional (criar projeto)

Endpoint: POST /api/ai/project-onboarding

```
Turno 1: messages=[]
-> IA pergunta sobre publico-alvo
Turno 2: messages=[ai, user]
-> IA pergunta sobre funcionalidades
Turno 3: messages=[ai, user, ai, user]
-> IA pergunta sobre prazo
Turno 4: messages=... (3+ user turns)
-> Backend forca: gere JSON com 4-6 historias
-> IA retorna { done: true, stories: [...] }
```

Feature B: Gerar histórias (US8)

Endpoint: POST /api/ai/generate-stories

User digita: "Sistema de delivery"

Prompt manda Gemini gerar JSON:

```
{ "stories": [{ "title": "Como cliente, quero...", "story_points": 3, "label": "feature" }]
}
```

User vê lista, marca quais quer, click "Adicionar".

Feature C: Resumo de sprint (US9)

Endpoint: POST /api/ai/sprint-summary

Backend calcula métricas locais (totalPoints, donePoints, byStatus) e manda pro Gemini com **todas** as histórias do sprint. Gemini retorna markdown com:

- **Entregas:** o que foi concluído
- **Gargalos:** ex "3 tarefas paradas em revisão por mais de 2 dias"
- **Sugestões:** 2-3 melhorias acionáveis

Segurança da IA: chave Gemini APENAS no .env do backend. Frontend nunca conhece. Toda chamada exige JWT válido + membership.

```

sprint-planner/
├── CLAUDE.md                <- contexto do projeto (que IA lê)
├── DEPLOY.md                <- passo a passo de deploy
├── APRESENTACAO.md         <- slides em Marp
├── docs/
│   ├── commits.md          <- Mermaid de cada commit (~30)
│   └── class-diagram.md    <- 6 diagramas de classes
├── backend/                <- API Express
│   ├── .env                <- secrets (gitignored)
│   ├── src/
│   │   ├── app.js          <- Express config (sem listen)
│   │   ├── server.js       <- listen para dev local
│   │   ├── database.js     <- cliente Supabase
│   │   ├── middleware/auth.js <- valida JWT + user existe
│   │   └── routes/
│   │       ├── auth.js     <- register/login (bcrypt + JWT)
│   │       ├── projects.js <- CRUD projetos + convites
│   │       ├── invitations.js <- aceitar/rejeitar convites
│   │       ├── stories.js  <- CRUD historias + move-to-sprint + status
│   │       ├── sprints.js  <- CRUD sprints
│   │       ├── dashboard.js <- metricas + velocity
│   │       └── ai.js       <- 3 endpoints Gemini
│   └── package.json
├── api/index.js            <- handler serverless Vercel
├── vercel.json             <- config build + rewrites
├── frontend/              <- React + Vite
│   ├── src/
│   │   ├── main.tsx        <- entry React
│   │   ├── App.tsx         <- raiz: auth + navegacao
│   │   ├── api.ts          <- apiFetch helper
│   │   ├── components/
│   │   │   ├── Avatar.tsx
│   │   │   ├── ConfirmModal.tsx
│   │   │   ├── Skeleton.tsx
│   │   │   └── StoryDetailPanel.tsx
│   │   └── pages/
│   │       ├── Login.tsx, Register.tsx
│   │       ├── Projects.tsx, CreateProject.tsx
│   │       ├── ProjectWorkspace.tsx
│   │       ├── Backlog.tsx, StoryForm.tsx
│   │       ├── Sprints.tsx, SprintForm.tsx
│   │       ├── KanbanBoard.tsx
│   │       ├── Dashboard.tsx
│   │       ├── MembersModal.tsx
│   │       ├── InvitationsPanel.tsx
│   │       └── AiGenerateModal.tsx

```

```
| | | └─ AiSummaryModal.tsx
| | └─ .env.production      <- VITE_API_URL (path relativo)
|
|─ .githubhooks/pre-push    <- auto-deploy Vercel no push
|─ scripts/setup.sh         <- ativa hooks pos-clone
|─ .gitignore               <- ignora .env, node_modules
```

8. DEPLOY — Como funciona em produção

Local (desenvolvimento)

```
# Terminal 1 - backend
cd backend && npm run dev    # roda na porta 3001

# Terminal 2 - frontend
cd frontend && npx vite      # roda na porta 5173
```

Frontend chama `http://localhost:3001/api/*` (CORS aberto pra dev).

Produção (Vercel)

1. **Frontend:** Vite builda para `frontend/dist/` → CDN da Vercel
2. **Backend:** vira serverless function em `api/index.js`
3. **Mesma URL** (sem CORS): `/api/auth/login` é roteado internamente pela Vercel
4. **Env vars:** SUPABASE_URL, SUPABASE_KEY, JWT_SECRET, GEMINI_API_KEY no vault encrypted da Vercel

Auto-deploy a cada `git push`

O hook `.githubhooks/pre-push` :

1. Roda `vercel --prod --yes` antes do push
2. Se deploy passar, push prossegue
3. Se falhar, push é cancelado

9. CONCEITOS-CHAVE QUE VOCÊ PRECISA ENTENDER

1. JWT (JSON Web Token)

- "Carteira de identidade" digital assinada
- Frontend salva no `localStorage` e envia em todo `fetch`
- Backend verifica a assinatura — se mexerem no token, rejeita
- Expira em 7 dias

2. Hash `bcrypt`

- Função matemática **irreversível** que transforma senha em string
- "minha-senha-123" → "\$2a\$10\$xKqL..." (60 chars)
- Mesmo dump do banco não revela senhas
- Login: compara hash do banco com hash da senha digitada

3. REST API

- HTTP verbs: `GET` (ler), `POST` (criar), `PUT` (atualizar), `DELETE` (remover)

- URLs como nomes: `/api/projects/5/stories` = "histórias do projeto 5"
- Resposta sempre em JSON

4. React State

- Componentes re-renderizam quando `useState` muda
- `useEffect(() => {...}, [deps])` roda quando deps mudam
- `props` passam dados de pai pra filho; `callbacks` passam eventos de filho pro pai

5. Serverless Function

- Em vez de servidor 24/7, código roda só quando alguém chama
- Cold start: primeira chamada após inatividade é mais lenta
- Vercel cuida de scaling automático

6. Optimistic Update

- Atualiza UI **antes** da API responder
- Se API falhar, reverte ao estado anterior
- Sensação de instantaneidade

10. 9 USER STORIES IMPLEMENTADAS

#	História	Onde está
US1	Criar conta e fazer login	Login.tsx, Register.tsx, routes/auth.js
US2	Criar projeto e convidar membros	MembersModal.tsx, routes/projects.js, routes/invitations.js
US3	Adicionar histórias ao backlog	Backlog.tsx, StoryForm.tsx, routes/stories.js
US4	Criar sprint com datas	Sprints.tsx, SprintForm.tsx, routes/sprints.js
US5	Mover histórias backlog ↔ sprint	<select> no Backlog, PUT /stories/:id/move-to-sprint
US6	Board Kanban com 4 colunas	KanbanBoard.tsx (drag & drop)
US7	Dashboard com burndown	Dashboard.tsx (recharts)
US8	Gerar histórias via IA	AiGenerateModal.tsx, POST /api/ai/generate-stories
US9	Resumo automático de sprint	AiSummaryModal.tsx, POST /api/ai/sprint-summary

11. ROTEIRO PARA DEMONSTRAR

Se for apresentar, siga essa ordem:

1. **Tela de login** — explica auth com bcrypt + JWT
2. **Criar projeto com wizard IA** — destaca 4-turnos conversacionais
3. **Backlog populado pela IA** — mostra que PO não encara tela vazia
4. **Convidar segundo usuário** — mostra fluxo completo (sino, aceitar)
5. **Criar sprint, iniciar, mover histórias** — <select> confiável
6. **Kanban com drag & drop** — destaque o "feedback instantâneo"
7. **Editar história no painel slide-out** — auto-save inline
8. **Resumo IA do sprint** — markdown estruturado, gerado em ~5s

12. DÚVIDAS FREQUENTES

"E se o Gemini cair?" US8 e US9 quebram. Outras features funcionam normal. Em produção real teríamos fallback ou modelo local.

"Por que SQLite e depois Supabase?" SQLite era pra dev local rápido. Migramos pra Supabase pra deploy real (banco na nuvem, acessível pela serverless function da Vercel).

"Por que Vercel?" Frontend estático (CDN) + serverless backend + free tier. Mesma URL = sem CORS.

"Como vocês usaram IA pra construir?" Claude Code (CLI da Anthropic) gerando código, refatorando, criando testes. 4 sub-agentes em paralelo implementando US4-US7. ~30 commits documentados em Mermaid.

"O que é otimização otimista?" No Kanban, quando você arrasta um card, a UI move ANTES da API confirmar. Se falhar, volta. Sentido instantâneo mesmo com latência.

"Como protegem a chave da IA?" Apenas em `.env` do backend (gitignored) e Vercel vault encrypted. Frontend nunca vê. Endpoint `/api/ai/*` é proxy autenticado.

Para aprofundar

- Banco de dados: `docs/class-diagram.md`
- Diagramas de cada commit: `docs/commits.md`
- Apresentação em slides: `APRESENTACAO.md`
- Deploy: `DEPLOY.md`
- Estrutura técnica completa: `CLAUDE.md`